

CSc 8830 Assignment 6: Optical Flow, Tracking and Structure from Motion

Liberty Ikpeogu

October 5, 2026

Submission

Item	Details
Course	CSc 8830: Computer Vision, Assignment 6
Author	Liberty Ikpeogu
Code	github.com/Librey/csc8830-computer-vision , folder module6/
Language	Python 3 (OpenCV, NumPy, Matplotlib, pandas)
Demo video	Screen recording (3 min 34 s), uploaded with this report in Google Classroom: runs optical_flow.py, plays the traffic flow video, runs lk_validate.py and sfm_book.py, and shows the 3D reconstruction

Each script carries a README block at its top with install and run commands.

Part A.1: What optical flow tells us

Optical flow recovered six kinds of information from my two videos: moving regions, object direction and speed, camera heading, relative depth, scene activity, and the places where flow fails. I computed dense flow with Farnebäck's method in OpenCV on a 30-second segment of each video, at half resolution.

Video	Segment	Size, frame rate	Camera
Traffic_static_shot.mp4	0:05 to 0:35	960×540, 29.97 fps	fixed on an overpass
Driving_pov.mp4	0:10 to 0:40	640×360, 25 fps	dashcam in a moving car

The flow videos (Traffic_static_shot_flow.mp4, Driving_pov_flow.mp4) show three panels: the original frame, the flow in HSV (hue encodes direction, brightness encodes speed), and the flow as arrows. flow_analysis.py produces every figure and number below.

1. Which pixels move: segmentation

With a fixed camera the background has zero flow, so any pixel moving more than 1 px/frame belongs to a moving object. At $t = 20$ s, 4.15% of the traffic frame passes that threshold. Grouping those pixels into connected regions gives 14 moving regions, and the regions sit on the cars.

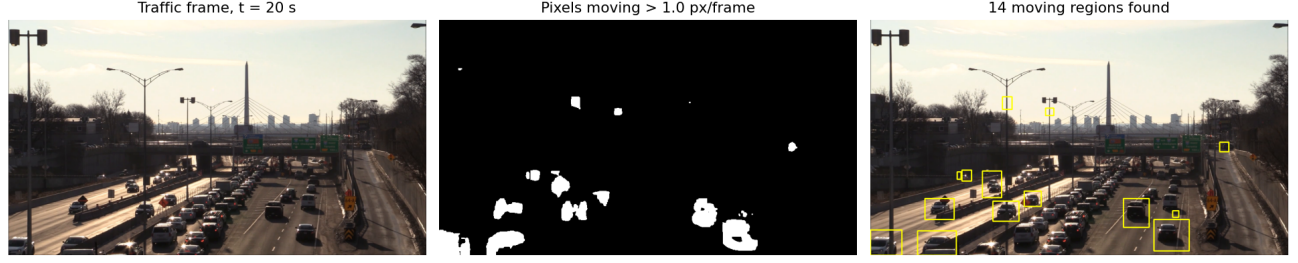


Figure 1: Thresholding the flow magnitude separates moving cars from the road, sky and bridge.

The mask marks motion. A car stopped in the jam on the right disappears from it, and a few small specks near the skyline come from noise in static pixels.

2. Direction and speed of each object

I split the traffic frame at the median strip and plotted the direction of every moving pixel. The two carriageways produce two separate direction peaks.

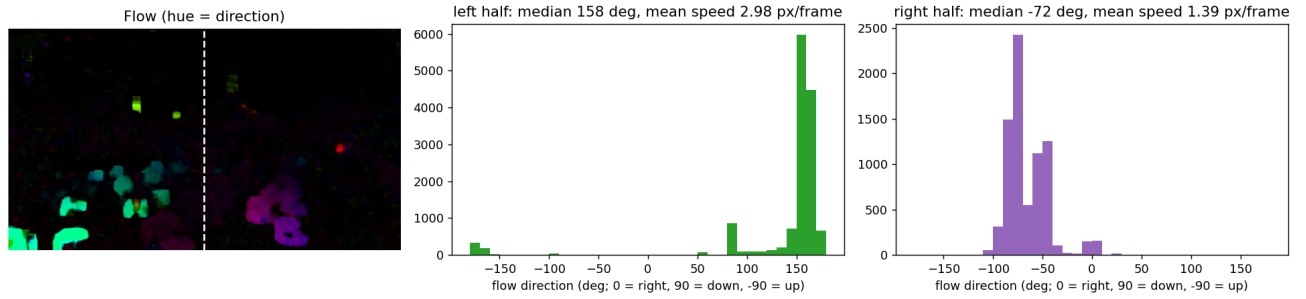


Figure 2: Flow direction histograms for the left and right halves of the traffic frame.

- **Left carriageway:** median direction 158° , leftward with a small downward component, so these cars approach the camera. Mean speed 2.98 px/frame.
- **Right carriageway:** median direction -72° , up the image, so these cars drive away from the camera. Mean speed 1.39 px/frame.

The flow tells you the lanes carry traffic in opposite directions, and that the right side moves at half the image speed of the left. Pixel speed depends on distance as well as true speed, so converting to km/h needs the camera calibration and road geometry.

3. How the camera moves: the focus of expansion

When the camera translates forward, every scene point flows outward along a line from one image point, the focus of expansion (FOE), which marks the direction of travel. Each flow vector $\mathbf{d}$ at pixel $\mathbf{p}$ defines a line through $\mathbf{p}$. With unit normal $\mathbf{n} = (-d_y, d_x)/|\mathbf{d}|$, a point $\mathbf{x}$ on that line satisfies:

$$\mathbf{n}_i^\top \mathbf{x} = \mathbf{n}_i^\top \mathbf{p}_i, \quad \hat{\mathbf{x}}_{FOE} = \arg \min_{\mathbf{x}} \sum_i w_i (\mathbf{n}_i^\top \mathbf{x} - \mathbf{n}_i^\top \mathbf{p}_i)^2$$

Stacking one row per moving pixel, weighted by $w_i = \sqrt{|d_i|}$, gives an overdetermined system that least squares solves.



Figure 3: Flow vectors and the estimated FOE at $t = 25$ s (left), and the FOE in 14 frame pairs (right).

- At $t = 25$ s the FOE lands at (359, 175), on the horizon above the road, and 98.6% of the vectors point away from it. That confirms forward motion.
- Across 14 frame pairs the FOE height stays within a standard deviation of 6.9 px. Its horizontal position varies more (std 44 px): the road curves, and near-horizontal vectors pin down the FOE height better than its left-right position.

The traffic video shows no FOE because its background flow is zero, so the flow field separates camera motion from object motion.

4. Relative depth from parallax

For a camera moving straight ahead, a point at depth Z and image distance r from the FOE moves at a speed proportional to r/Z . Near objects move fast and far objects move little. I averaged the flow speed in five regions over 14 frame pairs.

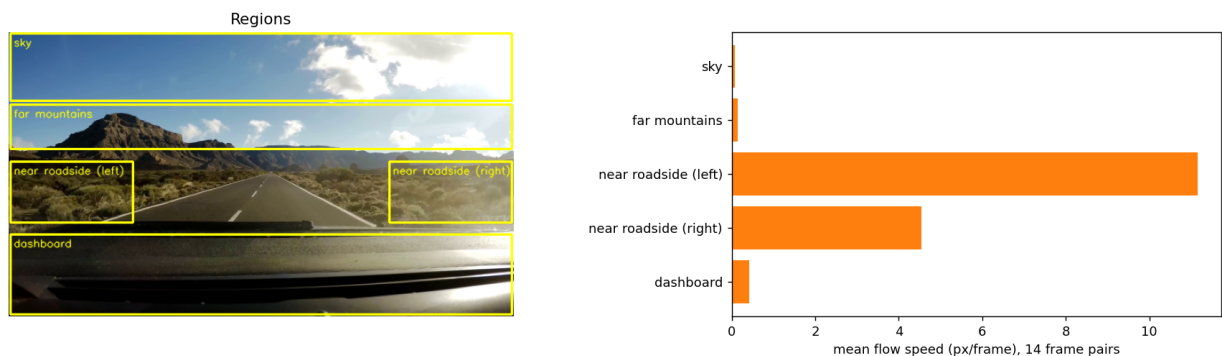


Figure 4: Mean flow speed per region in the driving video.

Region	Mean speed (px/frame)
Sky	0.07
Far mountains	0.14
Near roadside, right	4.54

Region	Mean speed (px/frame)
Near roadside, left	11.16
Dashboard	0.42

The roadside bushes move 30 to 80 times faster than the mountains, which ranks them as much closer. The left box sits lower in the frame and farther from the FOE than the right box, so its larger r gives it the higher speed. The dashboard travels with the camera; its 0.42 px/frame comes from vibration and reflections.

5. Scene activity over time

Two numbers per frame, mean flow speed and the share of pixels moving above 1 px/frame, separate the two videos for the whole 30 seconds.

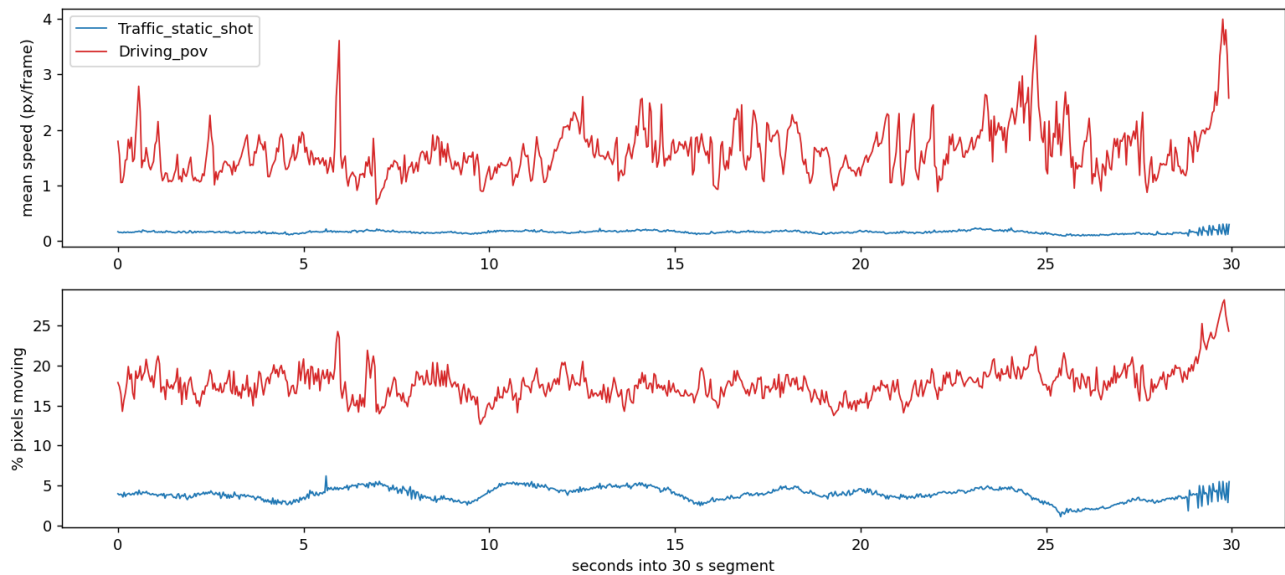


Figure 5: Flow statistics over each 30-second segment.

Video	Mean speed (px/frame)	Pixels moving
Traffic	0.16 (range 0.09 to 0.31)	3.9% (range 1.1 to 6.2)
Driving	1.62 (range 0.66 to 3.99)	17.7% (range 12.7 to 28.2)

The traffic curve stays flat because a steady stream of cars crosses a still background. The driving curve sits ten times higher and spikes when the car hits bumps, since a bump shakes the whole frame. These two numbers alone would classify a clip as fixed camera or moving camera.

6. Where optical flow fails

Flow needs texture. Part A.2 shows that each window yields a 2×2 system with matrix $A^T A$ built from image gradients; in a uniform region $A^T A$ has near-zero eigenvalues and the system has no reliable solution, which is the aperture problem.

To isolate texture from geometry, I compared two sets of pixels in one narrow strip around the centre lane line, in the same rows: the white dashes and the plain asphalt beside them. Both share the same depth, the same distance from the FOE, and so the same true motion.



Figure 6: Texture score (smallest eigenvalue of $A^T A$), the flow, and flow on dashes vs asphalt in the same strip.

Across 14 frame pairs the dashes average 3.08 px/frame and the asphalt 0.78 px/frame; the asphalt reads lower in 13 of the 14 pairs. Farnebäck smooths the field, so textureless asphalt inherits small values instead of its true motion. The sky reads 0.07 px/frame, but there distance and missing texture act together, so the flow alone cannot separate the two causes.

Part A.2: Motion tracking equations

Tracking a point between two frames reduces to a 2×2 linear system per point, $G d = e$, built from image gradients in a small window. I derive it below from one assumption about brightness, then set it up for two frames.

1. Brightness constancy

Let $I(x, y, t)$ be the image intensity. A scene point at pixel (x, y) at time t moves to $(x + \delta x, y + \delta y)$ at time $t + \delta t$. I assume the point keeps its brightness while it moves:

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

This holds for matte surfaces under steady lighting. Specular highlights, shadows and auto-exposure changes break it.

2. The optical flow constraint

For small δx , δy and δt , I expand the left side in a first-order Taylor series:

$$I(x + \delta x, y + \delta y, t + \delta t) \approx I(x, y, t) + \frac{\partial I}{\partial x} \delta x + \frac{\partial I}{\partial y} \delta y + \frac{\partial I}{\partial t} \delta t$$

Substituting into the constancy equation cancels $I(x, y, t)$:

$$I_x \delta x + I_y \delta y + I_t \delta t = 0$$

Dividing by δt and letting $\delta t \rightarrow 0$, with velocity $u = dx/dt$ and $v = dy/dt$, gives the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \quad \Leftrightarrow \quad \nabla I^\top \begin{bmatrix} u \\ v \end{bmatrix} = -I_t$$

The Taylor step drops second-order terms, so the equation holds only for motion of about one pixel or less. Section 5 handles larger motion.

3. The aperture problem

One pixel gives one equation in two unknowns. In the (u, v) plane the equation defines a line, and every point on that line satisfies it. The constraint fixes only the flow component along the gradient, the normal flow:

$$u_n = \frac{-I_t}{\|\nabla I\|}$$

The component along the edge stays unknown. Looking at a moving edge through a small aperture, you cannot tell whether it slides along itself.

4. Lucas-Kanade: a window of pixels

Lucas and Kanade add a second assumption: all pixels in a small window W around the point share the same motion $d = (u, v)$. A 5×5 window gives 25 equations in 2 unknowns:

$$\underbrace{\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ \vdots & \vdots \\ I_x(\mathbf{p}_n) & I_y(\mathbf{p}_n) \end{bmatrix}}_{A \ (n \times 2)} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} I_t(\mathbf{p}_1) \\ \vdots \\ I_t(\mathbf{p}_n) \end{bmatrix}}_{\mathbf{b}}$$

The system is overdetermined, so I minimise the squared error:

$$E(u, v) = \sum_{\mathbf{p} \in W} (I_x u + I_y v + I_t)^2$$

Setting $\partial E / \partial u = 0$ and $\partial E / \partial v = 0$:

$$\frac{\partial E}{\partial u} = 2 \sum I_x (I_x u + I_y v + I_t) = 0, \quad \frac{\partial E}{\partial v} = 2 \sum I_y (I_x u + I_y v + I_t) = 0$$

Rearranged, these give the normal equations $A^\top A d = A^\top b$:

$$\underbrace{\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}}_{G = A^\top A} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{\mathbf{e} = A^\top \mathbf{b}}$$

When G is invertible, Cramer's rule gives the closed form I use for the hand calculation in A.4:

$$\det G = \sum I_x^2 \sum I_y^2 - \left(\sum I_x I_y \right)^2$$

$$u = \frac{-\sum I_y^2 \sum I_x I_t + \sum I_x I_y \sum I_y I_t}{\det G}, \quad v = \frac{\sum I_x I_y \sum I_x I_t - \sum I_x^2 \sum I_y I_t}{\det G}$$

When the solution is reliable. G is the structure tensor of the window. Its eigenvalues $\lambda_1 \geq \lambda_2$ describe the texture:

Window content	Eigenvalues	Result
Flat region (asphalt, sky)	$\lambda_1 \approx \lambda_2 \approx 0$	no solution; G is singular
Straight edge (lane line)	λ_1 large, $\lambda_2 \approx 0$	aperture problem; only normal flow
Corner or rich texture	λ_1, λ_2 both large	unique, stable solution

Shi and Tomasi select points to track by requiring $\lambda_2 = \min(\lambda_1, \lambda_2)$ above a threshold. Figure 6 in A.1 plots that same λ_2 for the driving frame.

5. Setting up tracking between two frames

Given frame I at time t and frame J at time $t + 1$, and a point $p = (x_0, y_0)$ in I , I want the displacement d such that the window around p in I matches the window around $p + d$ in J :

$$J(\mathbf{x} + \mathbf{d}) \approx I(\mathbf{x}) \quad \text{for all } \mathbf{x} \in W(\mathbf{p}), \quad \mathbf{p}' = \mathbf{p} + \mathbf{d}$$

On a pixel grid with one frame step ($\delta t = 1$), I compute the derivatives with central differences in space and a frame difference in time:

$$I_x(x, y) = \frac{I(x+1, y) - I(x-1, y)}{2}, \quad I_y(x, y) = \frac{I(x, y+1) - I(x, y-1)}{2}, \quad I_t(x, y) = J(x, y) - I(x, y)$$

The tracking procedure for one point:

1. Select p in I where λ_2 of G exceeds a threshold (Shi-Tomasi).
2. Compute I_x, I_y over the window $W(p)$, and build G . G stays fixed during the iterations because it uses frame I only.
3. Start with $d_0 = (0, 0)$.
4. At iteration k , compute the temporal difference against the shifted window: $I_{t,k}(x) = J(x + d_k) - I(x)$. The position $x + d_k$ falls between pixels, so $J(x + d_k)$ needs bilinear interpolation (A.3).
5. Build $e_k = -\sum [I_x I_{t,k}, I_y I_{t,k}]^T$, solve $G \Delta d = e_k$, and update $d_{k+1} = d_k + \Delta d$.
6. Stop when $|\Delta d| < 0.01$ px or after 20 iterations. The tracked location is $p' = p + d$.

Each iteration is a Newton-Raphson step on E : the first-order expansion is redone around the current estimate, which removes the one-pixel limit of section 2 as long as the starting guess lands close enough. For motions of several pixels, an image pyramid runs the same steps from the coarsest level down, doubling d at each finer level. OpenCV's `calcOpticalFlowPyrLK` implements this pyramidal version, and A.4 compares my hand result against it.

Part A.3: Bilinear interpolation

Bilinear interpolation estimates the intensity at a non-integer position as a weighted average of the four surrounding pixels, with weights equal to the areas of the opposite rectangles. Tracking needs it because the shifted position $x + d_k$ in step 4 of A.2 almost never lands on a pixel centre.

1. Setup

To evaluate the image at a point (x, y) with non-integer coordinates, I take the pixel at its top-left corner and the fractional offsets:

$$x_0 = \lfloor x \rfloor, \quad y_0 = \lfloor y \rfloor, \quad a = x - x_0, \quad b = y - y_0, \quad 0 \leq a, b < 1$$

The four neighbours are $I_{00} = I(x_0, y_0)$, $I_{10} = I(x_0+1, y_0)$, $I_{01} = I(x_0, y_0+1)$ and $I_{11} = I(x_0+1, y_0+1)$.

2. Derivation from 1D linear interpolation

Between two samples $f(0)$ and $f(1)$, the straight line through both points gives:

$$f(a) = f(0) + a(f(1) - f(0)) = (1 - a)f(0) + af(1)$$

I apply this along x on the top row and on the bottom row:

$$I_{top} = (1 - a)I_{00} + aI_{10}, \quad I_{bottom} = (1 - a)I_{01} + aI_{11}$$

Then once along y between those two results:

$$I(x, y) = (1 - b)I_{top} + bI_{bottom}$$

Expanding gives the bilinear interpolation formula:

$$I(x, y) = (1 - a)(1 - b)I_{00} + a(1 - b)I_{10} + (1 - a)bI_{01} + abI_{11}$$

Interpolating along y first gives the same four terms, so the order does not matter. In matrix form:

$$I(x, y) = \begin{bmatrix} 1 - b & b \end{bmatrix} \begin{bmatrix} I_{00} & I_{10} \\ I_{01} & I_{11} \end{bmatrix} \begin{bmatrix} 1 - a \\ a \end{bmatrix}$$

3. Why the name “bilinear”

Collecting terms in a and b writes the same formula as a polynomial:

$$I(x, y) = c_0 + c_1 a + c_2 b + c_3 ab$$

$$c_0 = I_{00}, \quad c_1 = I_{10} - I_{00}, \quad c_2 = I_{01} - I_{00}, \quad c_3 = I_{11} - I_{10} - I_{01} + I_{00}$$

The function is linear in a with b held fixed, and linear in b with a held fixed. The ab term makes it a curved surface over the square, so it can pass through all four corner values, which a plane cannot do in general.

4. Properties

- **Weights as areas.** Each weight equals the area of the sub-rectangle opposite its pixel inside the unit square. The four areas sum to 1, so the result stays within the range of the four neighbours.
- **Exact at the samples.** At $a = b = 0$ the formula returns I_{00} , and likewise at the other corners.
- **Continuous.** Neighbouring squares share edge values, so the interpolated image has no jumps. Its gradient jumps at pixel boundaries, which causes no harm for tracking.

5. Worked example

Take $I_{00} = 100$, $I_{10} = 120$, $I_{01} = 80$, $I_{11} = 140$, and evaluate at $a = 0.3$, $b = 0.6$.

Pixel	Value	Weight	Weight $\times$ value
I_{00}	100	$(0.7)(0.4) = 0.28$	28.0
I_{10}	120	$(0.3)(0.4) = 0.12$	14.4
I_{01}	80	$(0.7)(0.6) = 0.42$	33.6
I_{11}	140	$(0.3)(0.6) = 0.18$	25.2
Total		1.00	101.2

The two-step route gives the same answer: $I_{\text{top}} = 0.7(100) + 0.3(120) = 106$, $I_{\text{bottom}} = 0.7(80) + 0.3(140) = 98$, and $0.4(106) + 0.6(98) = 101.2$. The polynomial form agrees too: $c_0 = 100$, $c_1 = 20$, $c_2 = -20$, $c_3 = 40$, and $100 + 20(0.3) - 20(0.6) + 40(0.18) = 101.2$.

6. Role in tracking

At each Lucas-Kanade iteration, every pixel x in the window needs $J(x + d_k)$. With $d_k = (0.37, -1.82)$, for example, the pixel $(120, 85)$ maps to $(120.37, 83.18)$, and its value comes from the formula above with $x_0 = 120$, $y_0 = 83$, $a = 0.37$, $b = 0.18$. Rounding to the nearest pixel instead would snap d to whole pixels and stall the sub-pixel refinement.

Part A.4: Validating tracking on real frames

The tracking equations from A.2 predict where 12 points move between two consecutive frames, and the predictions land within 0.42 px of the measured locations; the mean error is 0.16 px for the traffic video and 0.21 px for the driving video.

Method

I took frames 599 → 600 of the traffic video ($t = 20$ s) and frames 625 → 626 of the driving video ($t = 25$ s), both at half resolution. In each frame pair I selected six Shi-Tomasi corners: on moving cars for the traffic video, and from the far hillside to the near roadside for the driving video. `lk_validate.py` tracks every point three independent ways:

Method	What it uses
My LK (theory)	The A.2 equations coded in NumPy: central differences, $G d = e$, Newton iterations, A.3 bilinear interpolation, 3-level pyramid, 15×15 window
Measured location	Normalised cross-correlation of a 21×21 patch from frame t over a ± 20 px search in frame $t+1$, with a parabola fit for sub-pixel accuracy; no gradients and no LK assumptions
OpenCV	<code>cv2.calcOpticalFlowPyrLK</code> , same window and pyramid depth, as a reference implementation

The measured location serves as the actual pixel position: it finds where the same patch of pixels appears in the next frame by direct comparison.

Traffic video: frames 599 → 600

Pt	p (frame t)	d from my LK	p' predicted	p' measured	Iterations	NCC	Error vs measured (px)	Error vs OpenCV (px)
1	(271, 386)	(−1.43, 0.39)	(269.57, 386.39)	(269.66, 386.29)	2	0.97	0.139	0.002
2	(293, 443)	(−2.34, 1.05)	(290.66, 444.05)	(290.71, 443.99)	3	0.99	0.088	0.002
3	(218, 357)	(−1.35, −0.03)	(216.65, 356.98)	(216.70, 356.99)	2	0.96	0.045	0.007
4	(665, 482)	(0.44, −1.41)	(665.44, 480.59)	(665.38, 480.72)	3	0.96	0.145	0.001
5	(329, 451)	(−2.01, 0.97)	(326.99, 451.97)	(327.26, 452.01)	15	0.85	0.274	0.044
6	(177, 424)	(−2.02, 1.43)	(174.97, 425.43)	(175.07, 425.17)	8	0.91	0.275	0.011

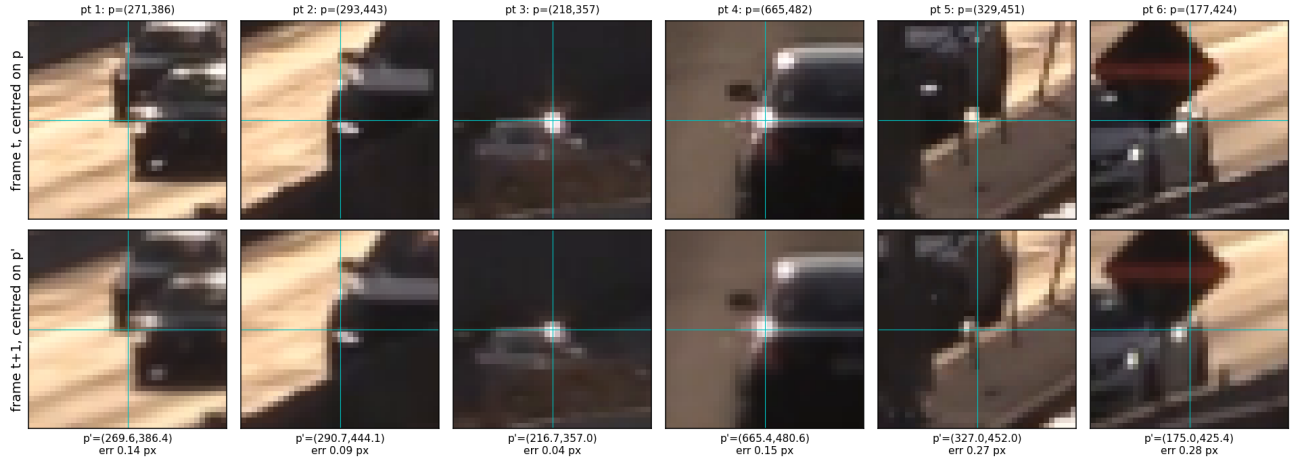


Figure 7: Traffic: each column centres frame t on p (top) and frame $t+1$ on the predicted p' (bottom). The same feature sits on the crosshair in both.

Points 1 to 4 sit on headlights and car edges with sharp texture and match within 0.15 px. Points 5 and 6 straddle a car and the road behind it; the window mixes two motions, which breaks the constant-motion assumption, so the solver needs more iterations and lands 0.27 px off.

Driving video: frames 625 → 626

Pt	p (frame t)	d from my LK	p' predicted	p' measured	Iterations	NCC	Error vs measured (px)	Error vs OpenCV (px)
1	(189, 163)	(0.00, 0.22)	(189.00, 163.22)	(188.95, 163.18)	2	0.99	0.061	0.004
2	(215, 200)	(-7.20, 1.34)	(207.80, 201.34)	(207.70, 201.74)	6	0.84	0.415	0.004
3	(172, 210)	(-12.95, 2.09)	(159.05, 212.09)	(158.87, 212.10)	8	0.63	0.176	0.017
4	(142, 212)	(-14.12, 2.54)	(127.88, 214.54)	(127.84, 214.13)	5	0.63	0.414	0.048
5	(119, 157)	(-0.17, 0.28)	(118.83, 157.28)	(118.88, 157.21)	2	0.98	0.086	0.005
6	(558, 153)	(0.11, -0.26)	(558.11, 152.75)	(558.01, 152.82)	2	1.00	0.126	0.008

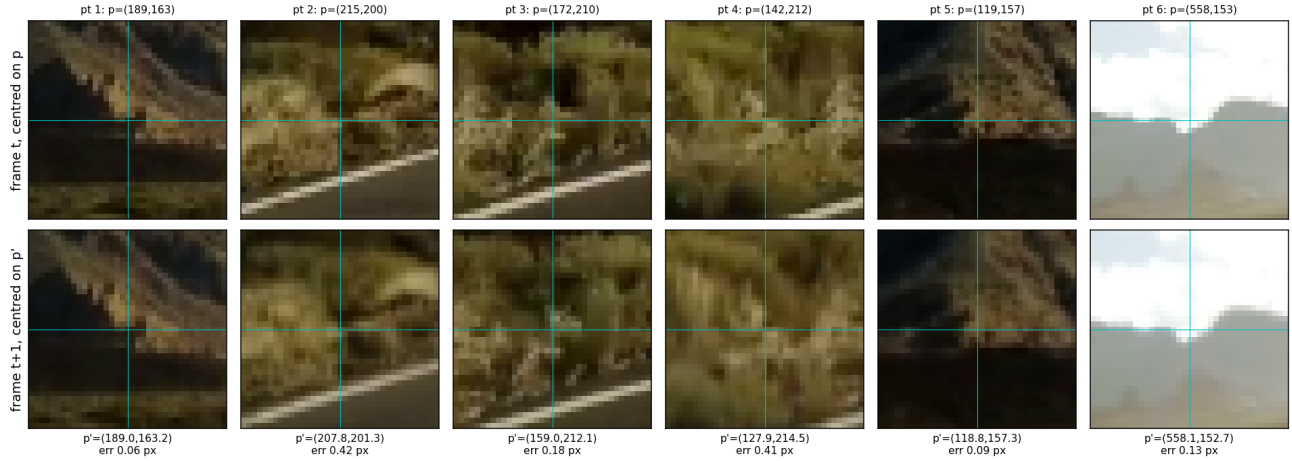


Figure 8: Driving: the far points (1, 5, 6) move under 0.3 px; the near roadside points (2, 3, 4) move 7 to 14 px and still land on the same bush.

The driving pair confirms the parallax result from A.1: hillside points move 0.2 to 0.3 px while roadside points move 7 to 14 px. A single Taylor step cannot reach 14 px, so the pyramid does the work: the coarsest level, at quarter resolution, sees a 3.5 px motion and passes its estimate down. The near points blur in motion, which drops the patch-matching score (NCC 0.63) and makes the measured location itself less certain; their 0.18 to 0.42 px gaps fall within that uncertainty.

Summary

Video	Points	Mean error vs measured (px)	Max error vs measured (px)	Mean error vs OpenCV (px)
Traffic	6	0.16	0.28	0.011
Driving	6	0.21	0.42	0.014

My implementation of the derived equations agrees with OpenCV to about 0.01 px, which confirms the code follows the same mathematics, and both agree with the measured locations to a fraction of a pixel.

Worked example: one point by hand

Point 3 of the traffic video, $p = (218, 357)$, sits on a car headlight. I use a 5×5 window at full half-resolution (no pyramid) so every number fits on the page. The intensities in the window, frame t and frame $t+1$:

$$I = \begin{bmatrix} 65 & 70 & 95 & 78 & 67 \\ 76 & 162 & 215 & 189 & 89 \\ 148 & 233 & 251 & 246 & 128 \\ 118 & 222 & 251 & 237 & 137 \\ 76 & 136 & 222 & 198 & 112 \end{bmatrix} \quad J = \begin{bmatrix} 51 & 55 & 54 & 48 & 40 \\ 162 & 188 & 116 & 51 & 32 \\ 245 & 255 & 185 & 84 & 46 \\ 227 & 248 & 168 & 114 & 65 \\ 122 & 162 & 124 & 103 & 71 \end{bmatrix}$$

The bright headlight (251) sits at the centre of I and has shifted left in J . Central differences on I and the frame difference give:

$$I_x = \begin{bmatrix} 8.5 & 15 & 4 & -14 & -10 \\ 55 & 69.5 & 13.5 & -63 & -70 \\ 58 & 51.5 & 6.5 & -61.5 & -95 \\ 64 & 66.5 & 7.5 & -57 & -77 \\ 33.5 & 73 & 31 & -55 & -61 \end{bmatrix} \quad I_y = \begin{bmatrix} 15.5 & 54 & 73.5 & 69.5 & 19.5 \\ 41.5 & 81.5 & 78 & 84 & 30.5 \\ 21 & 30 & 18 & 24 & 24 \\ -36 & -48.5 & -14.5 & -24 & -8 \\ -25 & -76 & -76.5 & -69.5 & -29.5 \end{bmatrix}$$

$$I_t = J - I = \begin{bmatrix} -14 & -15 & -41 & -30 & -27 \\ 86 & 26 & -99 & -138 & -57 \\ 97 & 22 & -66 & -162 & -82 \\ 109 & 26 & -83 & -123 & -72 \\ 46 & 26 & -98 & -95 & -41 \end{bmatrix}$$

Summing the element-wise products over the 25 pixels:

Sum	Value
ΣI_x^2	68 292.2
$\Sigma I_x I_y$	-6 024.5
ΣI_y^2	61 473.0
$\Sigma I_x I_t$	70 914.0
$\Sigma I_y I_t$	-14 634.0

$$G = \begin{bmatrix} 68292.2 & -6024.5 \\ -6024.5 & 61473.0 \end{bmatrix}, \quad \mathbf{e} = \begin{bmatrix} -70914.0 \\ 14634.0 \end{bmatrix}$$

Both eigenvalues of G are large (57 960 and 71 806), so the point is a corner and the system is well conditioned. With Cramer's rule from A.2:

$$\det G = 68292.2 \times 61473.0 - (-6024.5)^2 = 4.1618 \times 10^9$$

$$u = \frac{-61473.0 \times 70914.0 + (-6024.5)(-14634.0)}{4.1618 \times 10^9} = -1.026, \quad v = \frac{(-6024.5)(70914.0) - 68292.2 \times 14634.0}{4.1618 \times 10^9}$$

The first step moves the point 1.03 px left. The true motion exceeds one pixel, so the Taylor approximation undershoots, and the iterations of A.2 step 4 to 6 refine it. Each iteration re-samples J at the shifted window with bilinear interpolation:

Iteration	u (px)	v (px)	Step size $\ \Delta d\ $ (px)
1	-1.0263	0.1375	1.0354
2	-1.3255	0.0322	0.3172
3	-1.3495	0.0351	0.0242
4	-1.3512	0.0346	0.0017

One interpolation from iteration 2, for the window's top-left pixel (216, 355) shifted by d_1 : $J(214.9737, 355.1375)$ has $x_0 = 214$, $y_0 = 355$, $a = 0.9737$, $b = 0.1375$, and neighbours

45, 54, 50, 75. The A.3 formula gives $(0.0263)(0.8625)(45) + (0.9737)(0.8625)(54) + (0.0263)(0.1375)(50) + (0.9737)(0.1375)(75) = 56.59$.

The iterations stop after step 4, below the 0.01 px threshold, at $d = (-1.351, 0.035)$. The tracked location is $p' = (216.649, 357.035)$. The measured displacement is $(-1.304, -0.014)$, so the hand calculation lands 0.068 px from the actual pixel location.

Part B: Structure from motion with four views

From four photos of a flat book cover, I recovered its four corners and boundary in 3D and the position of the camera for each photo. With only the 14.5 cm width used to set the scale, the reconstruction gives a length of 20.05 cm (true 20.0 cm, +0.2%) and corner angles between 89.5° and 90.4° .

Object, images and camera

The object is a hardcover book, 14.5 cm wide and 20.0 cm long, lying flat on a table. I name the cover corners TL (top of the spine), TR, BR and BL (bottom of the spine), so TL-TR is the 14.5 cm width and TR-BR the 20 cm length.

View	File	Where I stood
1	view1_overhead.jpeg	above the book, looking down
2	view2_sideways.jpeg	beside the book, book turned sideways in the frame
3	view3_spine.jpeg	on the spine side, looking across the cover
4	view4_front.jpeg	in front of the bottom edge, looking up the cover

Camera parameter	Value
Camera	Apple iPhone 17 Pro, rear camera
Focal length (35 mm equivalent)	24 mm
Native resolution	3024×4032 px
Resolution of the images used	1932×2576 px
Focal length f	1786.1 px
Principal point (c_x, c_y)	(966, 1288) px, image centre
Skew, lens distortion	0; the iPhone corrects distortion in its image pipeline

Focal length calculation. A 35 mm-equivalent focal length maps the image diagonal to the 43.27 mm diagonal of a 36×24 mm frame:

$$f_{full} = f_{eq} \frac{\sqrt{3024^2 + 4032^2}}{\sqrt{36^2 + 24^2}} = 24 \times \frac{5040}{43.27} = 2795.7 \text{ px}, \quad f = 2795.7 \times \frac{1932}{3024} = 1786.1 \text{ px}$$

$$K = \begin{bmatrix} 1786.1 & 0 & 966 \\ 0 & 1786.1 & 1288 \\ 0 & 0 & 1 \end{bmatrix}$$

Method

All points on the cover lie on one plane, so a homography links any two views of it. The pipeline in `sfm_book.py`:

1. Locate the four corners in view 1 by fitting a line to each cover edge and intersecting neighbouring lines.
2. Match SIFT features on the cover between view 1 and each other view, and fit a homography H_{1j} with RANSAC.
3. Transfer the view-1 corners into views 2 to 4 through H_{1j} .
4. Decompose each H_{1j} into the rotation R_j , translation t_j and plane normal n .
5. Triangulate the corners from all four views, then refine every pose and point together with bundle adjustment.
6. Fix the metric scale with the 14.5 cm width; keep the 20 cm length, the diagonals and the angles as checks.
7. Express the camera centres in a coordinate frame attached to the book.



Figure 9: SIFT matches that survive RANSAC between view 1 and each other view: 91, 41 and 49 inliers.

Step 1: Corners in view 1

In view 1 no side face of the book shows, so each cover edge separates red cover from dark background. Along each rough edge I sampled 300 cross-sections, took the strongest red-to-background transition with sub-pixel accuracy, fitted a line, and intersected neighbouring lines. The line fits have median residuals of 0.5 to 1.1 px. The cover corners are rounded, so the intersection gives the ideal sharp corner of the rectangle.

Corner	View 1 (px)	View 2 (px)	View 3 (px)	View 4 (px)
TL	(106.96, 107.57)	(1186.71, 613.03)	(64.38, 924.31)	(419.16, 727.31)
TR	(1724.76, 115.20)	(1720.86, 1246.48)	(736.23, 410.27)	(1333.05, 763.30)
BR	(1726.81, 2313.15)	(588.11, 2020.33)	(1899.36, 930.37)	(1707.05, 2046.37)
BL	(130.25, 2340.63)	(85.10, 1109.57)	(1426.40, 1769.12)	(35.47, 2061.99)

Step 2: Homographies between views

A homography maps a point x_1 on the plane in view 1 to its position x_j in view j , in homogeneous coordinates, up to scale:

$$\mathbf{x}_j \approx H_{1j} \mathbf{x}_1$$

RANSAC on the SIFT matches gives, for view 2 (median transfer error 1.28 px over 91 inliers):

$$H_{12} = \begin{bmatrix} 0.188428 & -0.494142 & 1197.658163 \\ 0.283835 & 0.110646 & 559.380091 \\ -0.000080 & -0.000093 & 1 \end{bmatrix}$$

Worked transfer of corner TL from view 1 to view 2:

$$H_{12} \begin{bmatrix} 106.957 \\ 107.567 \\ 1 \end{bmatrix} = \begin{bmatrix} 1164.658 \\ 601.640 \\ 0.981421 \end{bmatrix} \Rightarrow \mathbf{x}_2 = \left(\frac{1164.658}{0.981421}, \frac{601.640}{0.981421} \right) = (1186.71, 613.03)$$

In views 2 to 4 I used these transferred corners instead of running an edge detector. In those views the spine or the page block shows next to some cover edges, and an edge detector locks onto the outer edge of that side face, 5 to 25 px outside the cover's top surface. The homography maps the top-surface plane, so its transferred corners stay on that plane.



Figure 10: Measured corners (cyan outline) and the bundle-adjusted 3D corners projected back into each view (red +).

Step 3: Decomposing the homography into motion

For a plane $\mathbf{n}^T \mathbf{X} = d$ in the frame of camera 1, and camera j with rotation R and translation $\mathbf{t}$, a plane point $\mathbf{X}$ projects to $\mathbf{x}_1 = K\mathbf{X}$ and $\mathbf{x}_j = K(R\mathbf{X} + \mathbf{t})$. On the plane $\mathbf{n}^T \mathbf{X} / d = 1$, so:

$$R\mathbf{X} + \mathbf{t} = R\mathbf{X} + \mathbf{t} \frac{\mathbf{n}^T \mathbf{X}}{d} = \left(R + \frac{\mathbf{t} \mathbf{n}^T}{d} \right) \mathbf{X} \Rightarrow H \approx K \left(R + \frac{\mathbf{t} \mathbf{n}^T}{d} \right) K^{-1}$$

Removing the intrinsics gives the normalised homography. Its middle singular value equals the unknown scale, so dividing by it fixes the scale:

$$H_n = K^{-1} H_{12} K = \begin{bmatrix} 0.265434 & -0.403861 & -0.017974 \\ 0.386511 & 0.231021 & -0.032302 \\ -0.142386 & -0.166931 & 0.802618 \end{bmatrix}, \quad \sigma = (0.8487, 0.4862, 0.4163)$$

The decomposition (Faugeras and Lustman; OpenCV `decomposeHomographyMat`) returns four candidate $(R, t/d, n)$ triples. Two place all matched points behind camera 1 ($n^T K^{-1} x_1 < 0$) and I discard them. Of the two left, one has $n = (-0.47, -0.47, 0.75)$, the other $n = (-0.015, -0.031, 0.999)$. Views 3 and 4 also produce a normal close to $(0, 0, 1)$, and view 1 looks almost straight down, so the second solution is the physical one:

$$R_2 = \begin{bmatrix} 0.5472 & -0.8280 & -0.1224 \\ 0.7881 & 0.4604 & 0.4087 \\ -0.2820 & -0.3201 & 0.9044 \end{bmatrix}, \quad \frac{t_2}{d} = \begin{bmatrix} 0.0855 \\ -0.4754 \\ 0.7468 \end{bmatrix}, \quad \mathbf{n} = \begin{bmatrix} -0.0145 \\ -0.0311 \\ 0.9994 \end{bmatrix}$$

Checks: $\det R_2 = 1.000$ and $R_2^T R_2 = I$, and $R_2 + t_2 n^T / d$ reproduces H_n / σ_2 to six decimal places. The rotation angle between views 1 and 2 is 62.9° ; views 3 and 4 give 62.8° and 40.9° .

Step 4: Triangulation

With $d = 1$ the projection matrices are $P_1 = K[I \mid 0]$ and $P_j = K[R_j \mid t_j]$. For a point $X = (X, Y, Z, 1)$ seen at (x, y) through P with rows p_1, p_2, p_3 , the cross product $x \times PX = 0$ gives two linear equations per view:

$$x(p_3^T X) - p_1^T X = 0, \quad y(p_3^T X) - p_2^T X = 0$$

Stacking them gives $AX = 0$, solved by the right singular vector of A with the smallest singular value. Worked example for corner TL from views 1 and 2:

$$A = \begin{bmatrix} -1786.13 & 0 & -859.04 & 0 \\ 0 & -1786.13 & -1180.43 & 0 \\ -1039.60 & 1408.28 & 418.30 & 12.09 \\ -1217.25 & -606.23 & -1340.39 & 345.00 \end{bmatrix}$$

The singular values of A are $(3003.2, 2526.8, 334.5, 0.0)$, so the system has an exact solution. The last right singular vector is $(-0.2915, -0.4005, 0.6060, 0.6224)$; dividing by its fourth entry:

$$\mathbf{X}_{TL} = (-0.4683, -0.6435, 0.9737) \text{ (units of } d\text{)}$$

The point satisfies $n^T X = 1.000$, so it lies on the recovered plane, and it reprojects to $(106.96, 107.57)$ in view 1 and $(1186.71, 613.03)$ in view 2, matching the measured pixels. The full pipeline triangulates every corner from all four views (an 8×4 system per corner).

Step 5: Bundle adjustment

Each homography gives its own estimate of the plane normal, and the three disagree by about 3° , so the four-view triangulation reprojects up to 14.5 px from the measured corners. Bundle adjustment minimises the total reprojection error over the poses of views 2 to 4 and all 3D points at once:

$$\min_{\{R_j, \mathbf{t}_j\}, \{\mathbf{X}_i\}} \sum_{(i,j)} \|\mathbf{x}_{ij} - \pi(K(R_j \mathbf{X}_i + \mathbf{t}_j))\|^2$$

Here π divides by the third coordinate. Camera 1 stays fixed at $K[I \mid 0]$, and a penalty holds the length of $\mathbf{t}_2$ constant to remove the free scale. The problem has 378 observations (4 corners in 4 views, plus 181 SIFT points in two views each) and 573 unknowns. Levenberg-Marquardt cuts the maximum reprojection error from 14.5 px to 6.3 px, with a mean of 0.68 px.

Step 6: Metric scale and the recovered boundary

The reconstruction is known up to scale. The two width edges measure 0.9046 and 0.9066 units; scaling their mean to 14.5 cm gives $s = 16.01$ cm per unit, which also makes the distance from camera 1 to the cover 16.0 cm.

Measurement	Recovered	True	Error
Width TL-TR, BR-BL (cm)	14.48, 14.52	14.5	used for scale
Length TR-BR, BL-TL (cm)	20.09, 20.00	20.0	+0.05 cm mean (+0.2%)
Diagonals TL-BR, TR-BL (cm)	24.80, 24.68	24.70	+0.10, -0.02 cm
Corner angles TL, TR, BR, BL (deg)	89.95, 90.14, 89.47, 90.44	90	within 0.53°
SIFT surface points, distance from corner plane (cm)	median 0.07, 90th percentile 0.15	0	flat to 1.5 mm

The corners in the book frame (origin BL, x along the bottom edge, y up the spine, z out of the cover) are TL $(-0.15, 20.00, 0.00)$, TR $(14.33, 20.09, 0.17)$, BR $(14.52, 0.00, 0.00)$ and BL $(0, 0, 0)$ cm. For a boundary with more than four points, I back-projected the full cover outline from view 1 onto the recovered plane. It follows the corner rectangle and reproduces the rounded corners of the cover.

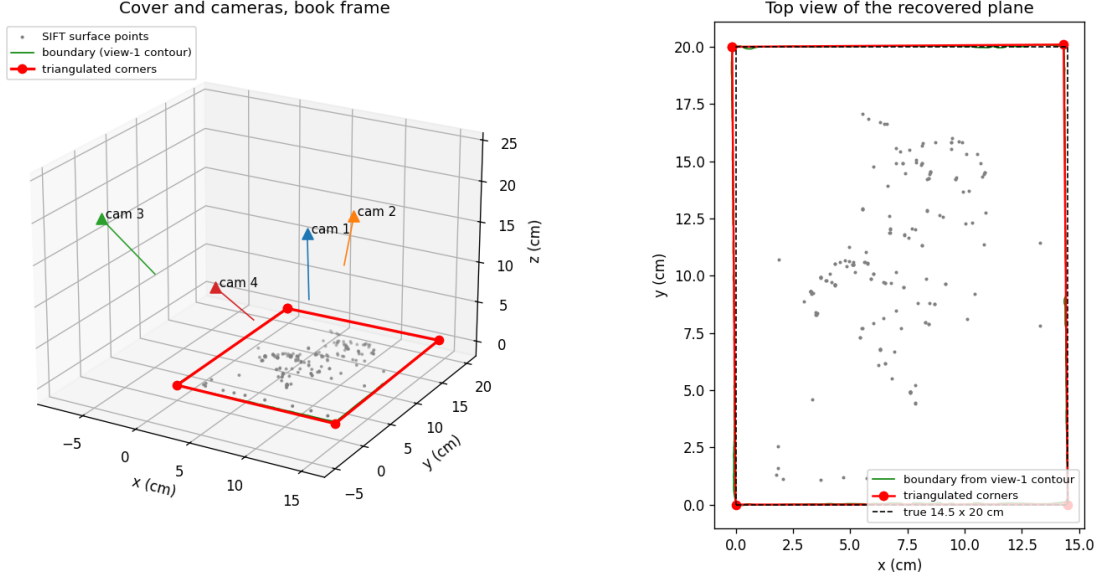


Figure 11: Left: recovered cover, SIFT surface points and camera positions in the book frame. Right: top view of the recovered boundary against the true 14.5×20 cm rectangle.

Step 7: Camera positions

A camera with pose (R, t) has its centre at $C = -R^T t$ in the frame of camera 1. Worked example for view 2, using the bundle-adjusted pose:

$$R_2^T t_2 = \begin{bmatrix} 0.5496 & 0.7880 & -0.2776 \\ -0.8263 & 0.4635 & -0.3201 \\ -0.1235 & 0.4052 & 0.9058 \end{bmatrix} \begin{bmatrix} 0.0871 \\ -0.4741 \\ 0.7475 \end{bmatrix} = \begin{bmatrix} -0.5332 \\ -0.5310 \\ 0.4742 \end{bmatrix}$$

$$C_2 = -R_2^T t_2 \times s = (0.5332, 0.5310, -0.4742) \times 16.01 = (8.54, 8.50, -7.59) \text{ cm (camera-1 frame)}$$

To move it into the book frame I subtract the BL corner, $(-7.50, 9.48, 16.07)$ cm in the camera-1 frame, and rotate by the matrix whose rows are the book axes. That gives $C_2 = (15.3, 1.2, 24.1)$ cm: 15.3 cm along the bottom edge, past the BR corner, and 24.1 cm above the cover.

View	Position x, y, z (cm)	Distance to cover centre (cm)	Tilt from straight down
1	(7.0, 9.6, 16.2)	16.2	1.7°
2	(15.3, 1.2, 24.1)	26.9	23.7°
3	(-7.7, 1.2, 17.6)	24.6	32.9°
4	(6.4, -5.0, 16.1)	22.0	40.8°

The positions match how I took the photos. Camera 1 sits above the cover centre (7.25, 10). Camera 3 sits at negative x, on the spine side. Camera 4 sits at negative y, in front of the bottom edge.

Limitations

- **Focal length.** K assumes all four photos used the 24 mm-equivalent setting shown for view 1. Solving for f from the known rectangle in each oblique view gives estimates between 1097 and 2120 px; at these tilts the rectangle gives only a loose constraint on f , so the right angles and 20 cm length recovered with $f = 1786$ px give the stronger confirmation.
- **Corner localisation.** Rounded corners and visible side faces limit the corner accuracy to a few pixels, which accounts for most of the 6.3 px maximum reprojection error.
- **Book thickness.** The method reconstructs the top surface of the cover only; the spine and page edges lie off that plane and do not enter the reconstruction.

References

Course material

1. S. K. Nayar, *First Principles of Computer Vision*, YouTube playlist “Optical Flow | Structure from Motion | Object Tracking”, Columbia University. [Playlist](#)

Optical flow and tracking (Part A)

2. B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” *Proc. 7th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 674–679, 1981.
3. B. K. P. Horn and B. G. Schunck, “Determining optical flow,” *Artificial Intelligence*, vol. 17, pp. 185–203, 1981.
4. G. Farnebäck, “Two-frame motion estimation based on polynomial expansion,” *Proc. 13th Scandinavian Conference on Image Analysis (SCIA)*, LNCS 2749, pp. 363–370, 2003.
5. J. Shi and C. Tomasi, “Good features to track,” *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 593–600, 1994.
6. J.-Y. Bouguet, “Pyramidal implementation of the affine Lucas Kanade feature tracker: description of the algorithm,” Intel Corporation, Microprocessor Research Labs, technical report, 2001.

Structure from motion (Part B)

7. D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
8. M. A. Fischler and R. C. Bolles, “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
9. O. Faugeras and F. Lustman, “Motion and structure from motion in a piecewise planar environment,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 2, no. 3, pp. 485–508, 1988.
10. E. Malis and M. Vargas, “Deeper understanding of the homography decomposition for vision-based control,” INRIA Research Report RR-6303, 2007.
11. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
12. B. Triggs, P. McLauchlan, R. Hartley and A. Fitzgibbon, “Bundle adjustment: a modern synthesis,” *Vision Algorithms: Theory and Practice*, LNCS 1883, pp. 298–372,

- 2000.
13. Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.

Software

14. OpenCV, Open Source Computer Vision Library. docs.opencv.org
15. NumPy, SciPy (scipy.optimize.least_squares) and Matplotlib, Python scientific libraries.

Data

16. Traffic_static_shot.mp4 and Driving_pov.mp4: free stock footage downloaded from the web.
17. Book photographs (Part B): taken by the author with an Apple iPhone 17 Pro, October 5, 2026.